

小程序及跨端框架中 onShow 和 onHide 生命周期钩子解析

一、基础概念与核心定位

`onShow` 和 `onHide` 是小程序（微信/支付宝/抖音等）、`uni-app`、`Taro` 等跨端框架中用于监听页面/组件**显示/隐藏状态**的生命周期钩子，核心作用是响应页面/组件的可见性变化，而非页面的创建/销毁。

适用场景区分

钩子	触发时机	核心用途
<code>onShow</code>	页面/组件 变为可见状态 时触发	刷新数据、恢复定时器、重新请求接口、播放视频/音频等
<code>onHide</code>	页面/组件 变为不可见状态 时触发	暂停定时器、停止播放媒体、保存临时数据、取消请求等

二、核心对比（维度拆解）

对比维度	onShow	onHide
触发条件	页面/组件从“隐藏” → “可见”	页面/组件从“可见” → “隐藏”
首次触发时机	页面初始化完成（ <code>onLoad</code> / <code>onReady</code> 后）首次显示时触发	首次显示后，页面被隐藏时才触发（首次加载不会触发）
触发频率	每次页面显示都会触发（包括返回、切换标签、小程序切回前台）	每次页面隐藏都会触发（包括跳转、切换标签、小程序切后台）
携带参数	无固定参数（可通过页面栈获取上下文）	无固定参数
与路由的关系	页面跳转返回（如 <code>navigateBack</code> ）、标签切换时触发	页面跳转（如 <code>navigateTo</code> ）、标签切换、小程序退后台时触发

组件层面支持	小程序自定义组件需用 <code>show / hide</code> 监听（而非 <code>onShow / onHide</code> ），uni-app 组件支持 <code>onShow / onHide</code>	同左
--------	---	----

三、执行顺序（核心场景示例）

场景1：页面初始化与首次显示

Code block

```
1  onLoad（页面加载） → onReady（页面渲染完成） → onShow（页面显示）
```

- 此时 `onHide` 不会触发，因为页面从未隐藏过。

场景2：页面A → 跳转至页面B

Code block

```
1  页面A：onHide（A隐藏） → 页面B：onLoad → onReady → onShow
```

- 先隐藏当前页面A，再加载并显示新页面B。

场景3：页面B → 返回至页面A

Code block

```
1  页面B：onUnload（B销毁） → 页面A：onShow（A重新显示）
```

- 注意：页面B是 `onUnload` 而非 `onHide`（`navigateTo` 跳转的页面返回时会销毁），页面A触发 `onShow`。

场景4：小程序切后台/前台

Code block

```
1  切后台：当前页面 onHide → 切前台：当前页面 onShow
```

- 小程序退到手机后台（如切到微信/相册），当前页面触发 `onHide`；切回小程序，触发 `onShow`。

场景5：tabBar 页面切换（如首页 ↔ 我的）

Code block

- 1 首页 → 我的：首页 onHide → 我的 onShow（若我的页面已加载过，不会触发 onLoad/onReady）
- 2 我的 → 首页：我的 onHide → 首页 onShow

- tabBar 页面切换时，页面不会销毁（仅隐藏），因此仅触发 `onHide` / `onShow`，不触发 `onLoad` / `onUnload`。

场景6：页面A → 重定向至页面C（redirectTo）

Code block

- 1 页面A：onUnload（A销毁）→ 页面C：onLoad → onReady → onShow

- `redirectTo` 会销毁当前页面，因此页面A触发 `onUnload` 而非 `onHide`，页面C正常初始化并显示。

四、关键注意事项

1. onShow 与 onLoad 的区别：

- `onLoad` 仅在页面**首次加载**时触发一次，携带路由参数；
- `onShow` 每次页面显示都触发，无路由参数（需通过 `getCurrentPages()` 获取）。
- 示例：tabBar 页面切换时，`onLoad` 不触发，`onShow` 触发。

2. 组件层面的差异：

- 微信小程序自定义组件：需通过 `lifetimes.show` / `lifetimes.hide` 监听，而非直接写 `onShow` / `onHide`；
- uni-app 组件：支持 `onShow` / `onHide`，与页面逻辑一致。

3. 常见坑点：

- 跳转页面时忘记在 `onHide` 暂停定时器，导致后台仍执行；
- 依赖 `onShow` 获取实时数据，但未处理异步请求的重复执行。

五、示例代码（uni-app/微信小程序）

Code block

- 1 // 页面逻辑
- 2 Page({

```

3   onLoad(options) {
4       console.log('页面加载', options); // 仅触发一次
5   },
6   onShow() {
7       console.log('页面显示'); // 每次显示都触发
8       this.refreshData(); // 刷新数据
9   },
10  onHide() {
11      console.log('页面隐藏');
12      this.pauseTimer(); // 暂停定时器
13  },
14  onUnload() {
15      console.log('页面销毁');
16  },
17  methods: {
18      refreshData() {
19          // 重新请求接口
20      },
21      pauseTimer() {
22          // 清除定时器
23          clearInterval(this.timer);
24      }
25  }
26  });
27
28  // 微信小程序组件逻辑
29  Component({
30      lifetimes: {
31          show() {
32              console.log('组件显示');
33          },
34          hide() {
35              console.log('组件隐藏');
36          }
37      }
38  });

```

总结

- 执行顺序核心：隐藏前页面 onHide → 新页面初始化/旧页面显示 → onShow；
- 用途核心：onShow 处理“恢复/刷新”逻辑，onHide 处理“暂停/保存”逻辑；
- 记忆口诀：“显用 onShow，隐用 onHide，加载用 onLoad，销毁用 onUnload”。

（注：文档部分内容可能由 AI 生成）